

Introduction

Currently, official ABAP tool support exists for SAP GUI and Eclipse. For years, users have asked us to bring this support to additional IDEs. Based on the user survey results from 2023 and 2025, the most requested development environment is Visual Studio Code (VS Code). However, many users have also expressed interest in other environments such as JetBrains IDEs, Neovim, or even Zed. In short, our user base wants more flexibility when choosing their development environment.

See related article: [ABAP Development Tools for VS Code: Everything You Need to Know](#)

How Do We Transform the ABAP Development Tools Architecture to Offer More Choice?

When analyzing the challenge, we identified two major issues that needed to be addressed:

1. The Client/Server Architecture

ABAP tools follow a client/server architecture. For each new development environment, a client layer implementation is required. This layer contains all the code needed to communicate with an ABAP server, including complex wrappers for existing ABAP development tools REST APIs. It ensures compatibility with system releases as far back as SAP NetWeaver 7.3 EHP1 SP04 and higher and implements client-side tool logic such as the debugger, test runner, ATC, tracing, and more. Overall, the client UI-independent (IDE independent) codebase consists of 2.9 million lines of code!

2. Object Type Editors

ABAP relies heavily on object type editors. For the SAP BTP ABAP development environment alone, 88 unique editors need to be supported as of 2025. For comparison, CAP only requires one unique editor (CDS), and Java tooling only needs editors for classes, interfaces and enumerations.

Solution 1: Reusing Our Client Codebase as a Language Server

Modern development environments support language servers through the [Language Server Protocol \(LSP\)](#), which defines a standardized interface between the development tool and the language server.



(Image [source](#))

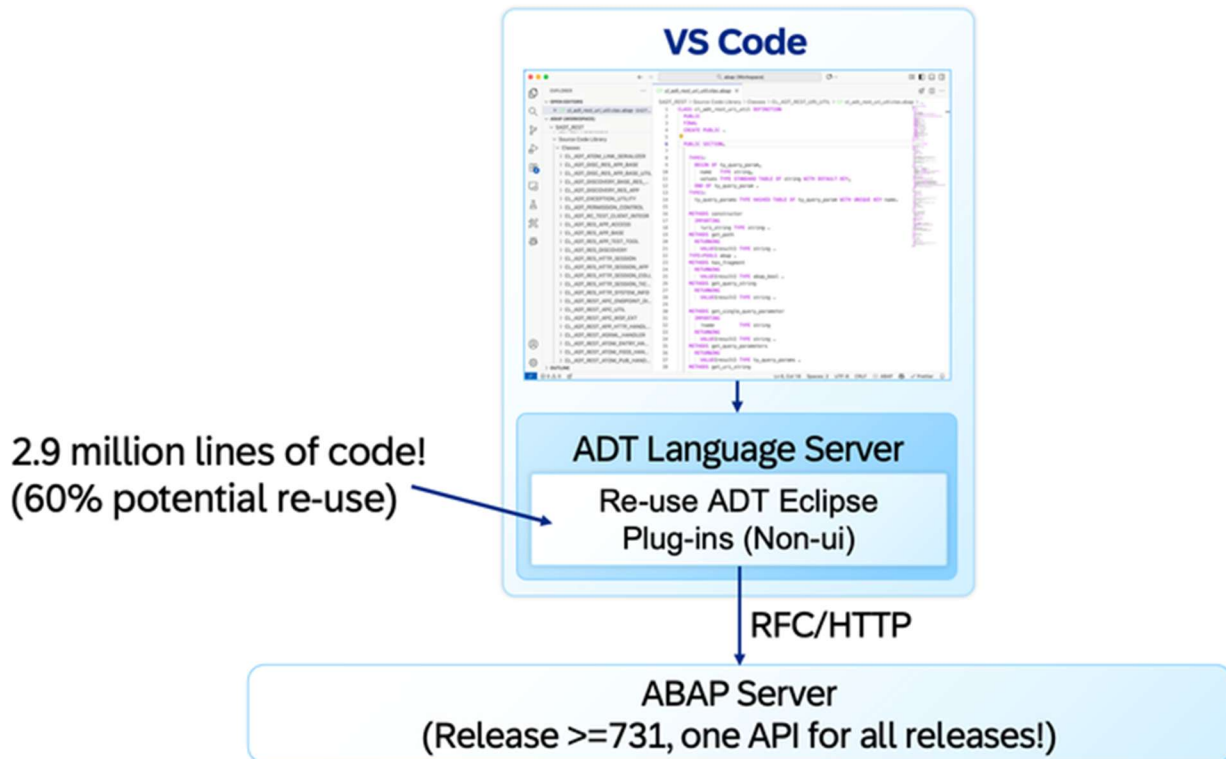
This means a language server can be written in any programming language and reused across multiple development environments.

We began exploring this approach in 2018, considering a TypeScript based language server to re-implement the client component—similar to community ABAP VS Code extensions. While functional, this approach was not ideal for us because it required rebuilding the entire client layer from scratch and maintaining two separate implementations. This is simply not a viable solution in the long run.

Fortunately, we discovered the [Java VS Code extension](#), which provided a clever solution: they "wrapped" the Eclipse Java development tools with a language server.

This approach allowed them to reuse the existing JDT codebase. Interestingly, most Java developers don't realize that the VS Code Java extension is essentially Eclipse running inside VS Code!

This concept solved our problem: we can potentially reuse 2.9 million lines of our client non-ui code.



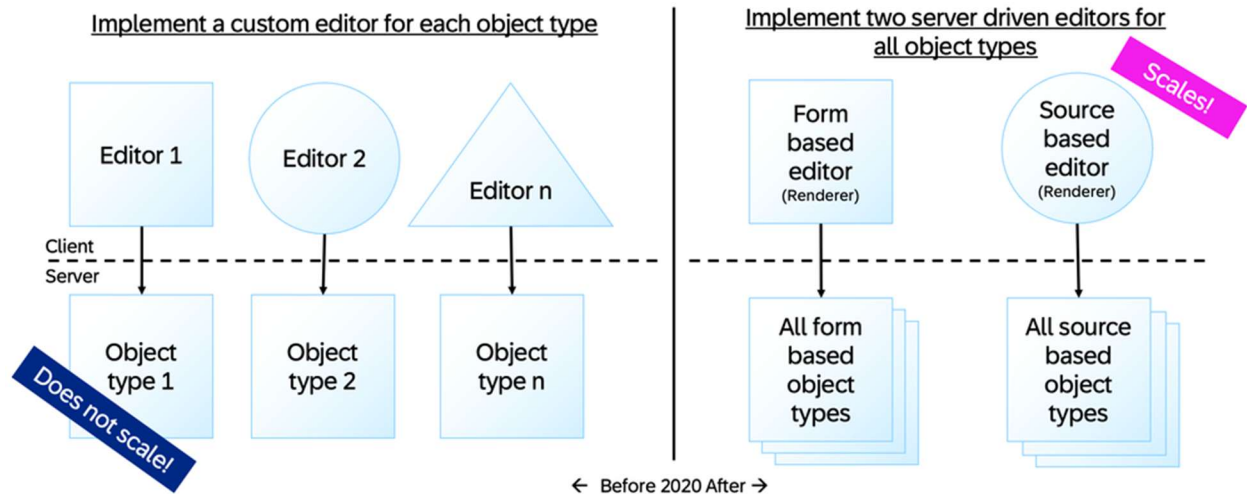
It also means we can maintain and support Eclipse and VS Code using the same codebase. Even better, users can connect to any ABAP server release supported by Eclipse through the VS Code extension.

Solution 2: Making Object Types Reusable via Server-Driven Development and Client-Based Renderers

Initially, when building the Eclipse plug-in, we implemented new object type editors by adding persistence logic on the server in ABAP and the UI on the client side in Java. In 2018, when the SAP BTP ABAP Environment was launched and we decided to only support ABAP Development Tools for Eclipse for ABAP Cloud development, we had to add many missing object types quickly. During this process, we realized our architecture did not scale well. Using two programming languages was too complex and slow.

Learning from past experiences, we looked at the server-side Dynpro and SAP Fiori Elements programming models, which allow the implementation of ABAP-based UIs using only ABAP code and a client-side rendering engine based on UI models.

Adopting this approach solved our scalability problem: we only need two rendering engines for tool support—one for form-based tools and one for source-based tools.



In 2020, we released editor support for number range objects—our first server-driven object type. Today, all new object types are built using our server-driven development framework.

Thanks to this shift, offering ABAP tool support in another development environment no longer requires 88 separate editors. It only requires two:

- One editor for form-based objects
- One editor for source-based objects

Conclusion

For years, we wanted to bring ABAP tools to more development environments, but the previous architecture made this impossible. Over the past six years, we have invested significant effort into re-architecting our tools. Thanks to this work, we can now bring ABAP tools to additional IDEs—starting with VS Code in 2026!